

第 3 章 分支和循环

前面写的程序都是从上到下一遍走完。真正有用的程序需要会拐弯和重复，条件不同走不同的路，同样的事重复很多遍。这一章做一个猜数字游戏，把这两件事讲透。

3.1 先做一个猜数字游戏

本节任务：电脑随机想一个 1 到 100 的数，你来猜，它提示大了还是小了，猜中就结束并报出用了几次。

```
1 import random
2
3 answer = random.randint(1, 100) # 随机想一个数
4 count = 0
5
6 while True: # 一直循环，直到猜中
7     guess = int(input("猜一个 1 到 100 的数: "))
8     count = count + 1
9
10    if guess == answer:
11        print(f"猜中了，你用了 {count} 次")
12        break # 跳出循环，游戏结束
13    elif guess < answer:
14        print("小了")
15    else:
16        print("大了")
```

运行起来是这样，冒号后面的数字是你敲进去的，这一局的答案是 68：

```
猜一个 1 到 100 的数: 50
小了
猜一个 1 到 100 的数: 75
大了
猜一个 1 到 100 的数: 62
小了
猜一个 1 到 100 的数: 68
猜中了，你用了 4 次
```

不到二十行就是一个能玩的游戏。它用到了循环让游戏反复进行，用分支判断大小，还用 break 在猜中时结束。下面逐个拆开。

3.2 分支，让程序会拐弯

本节任务：把条件判断的几种写法弄清楚。

上一章已经见过 if，这里补全。基本形式是 if 跟条件跟冒号，下面缩进四格：

```
1 score = 85
2 if score >= 60:
3     print("及格")
4 else:
5     print("不及格")
```

条件多于两种时用 **elif**，可以写很多个。Python 从上往下检查，谁先成立就执行谁，★核心剩下的一概跳过：

```
1 score = 85
2 if score >= 90:
3     print("优秀")
4 elif score >= 80:
5     print("良好") # 85 在这里停下
6 elif score >= 60:
7     print("及格")
8 else:
9     print("不及格")
```

因为是从上往下且只走一条，条件的顺序很重要。要是把 `score >= 60` 写在最前面，85 分就会被判成及格，后面的分支永远轮不到。

多个条件可以用 **and** 和 **or** 连起来，**not** 表示取反：

```
1 age = 20
2 if age >= 18 and age < 60:
3     print("成年人")
4 if age < 12 or age > 65:
5     print("半价")
6 if not (age >= 18):
7     print("未成年")
```

and 要求两边都成立，**or** 只要有一边成立。上面第一个判断还有更简洁的写法，Python 支持连着写：

```
1 if 18 <= age < 60: # 和 age >= 18 and age < 60 一个意思
2     print("成年人")
```

3.3 for 循环，把一批数据走一遍

本节任务：会用 for 遍历列表和数字区间。

要对一批数据每个都做点事，用 **for**。它的意思是把后面这一批东西一个一个取出来，每取一个执行一遍循环体：

```
1 scores = [86, 92, 78]
2 for s in scores:
3     print(s)
```

s 是个临时名字，每轮循环会自动换成下一个元素。名字可以随便取，取得有意义即可。

想循环固定的次数，用 **range**。range(5) 产生 0 到 4 这五个数，注意从 0 开始且不含终点。

```
1 for i in range(5):
2     print(i) # 依次输出 0 1 2 3 4
3
4 for i in range(1, 6):
5     print(i) # 依次输出 1 2 3 4 5
6
7 for i in range(0, 10, 2):
8     print(i) # 0 2 4 6 8 第三个参数是步长
```

不含终点这一点和切片一致。想数 1 到 5 就写 range(1, 6)。

遍历时如果既要元素又要序号，用 enumerate：

```
1 names = ["小明", "小红"]
2 for i, name in enumerate(names):
3     print(f"第 {i+1} 个是 {name}")
```

配合上一章的字典，遍历通讯录就是这样：

```
1 contacts = [{"name": "小明", "phone": "138"}, {"name": "小红", "phone": "139"}]
2 for person in contacts:
3     print(person["name"], person["phone"])
```

3.4 while 循环，不知道要转多少圈时用

本节任务：分清什么时候用 for 什么时候用 while。

for 适合已知要循环多少次或有一批数据。但猜数字游戏事先不知道要猜几次，这时用 **while**，它的意思是只要条件还成立就一直转：

```
1 count = 0
2 while count < 3:
3     print(count)
4     count = count + 1 # 别忘了改变条件，否则永远转不完
```

用 while 一定要注意，**循环体里必须有能让条件变假的操作**，否则就成了死循环，★核心程序卡住不动。上面那行 count 加一就是干这个的，去掉它程序就会一直打印 0。真遇到死循环，按 Ctrl 加 C 强行终止。

游戏里的写法是 while True，条件永远成立，也就是无限循环，靠里面的 break 跳出。这在不知道循环多少次、只知道满足某条件就结束时很常用。

break 是立刻跳出整个循环，**continue** 是跳过本轮剩下的部分直接进入下一轮：

```
1 for i in range(10):
2     if i == 5:
3         break # 到 5 就整个不循环了，输出 0 1 2 3 4
4     print(i)
5
6 for i in range(5):
7     if i % 2 == 0:
8         continue # 偶数跳过，只输出 1 3
9     print(i)
```

两种循环的分工如表 3.1。

表 3.1: for 和 while 怎么选。

什么时候用	
for	有一批数据要挨个处理，或者要循环固定次数
while	事先不知道多少次，只知道满足某个条件才停

3.5 把循环写短：推导式

本节任务：认识一种从旧列表造新列表的紧凑写法。

从一个列表加工出另一个列表很常见。老实写法是这样：

```
1 scores = [86, 92, 78]
2 plus = []
```

```
3 for s in scores:
4     plus.append(s + 5)
5 print(plus) # [91, 97, 83]
```

Python 提供了一个更短的写法叫**列表推导式**，一行做完同样的事：

```
1 plus = [s + 5 for s in scores]
2 print(plus) # [91, 97, 83]
```

读法是从中间的 for 开始，对 scores 里每个 s，把 s+5 收集起来。还可以在后面加条件，只收集满足条件的：

```
1 scores = [86, 92, 78, 55]
2 passed = [s for s in scores if s >= 60]
3 print(passed) # [86, 92, 78]
```

推导式在数据处理里用得非常多，写熟了比循环清爽。但也别嵌太多层，一行塞不下就老老实实写循环，可读性比短更重要。

3.6 回头看那个游戏

本节任务：把前面的知识对回到开头的代码上。

现在再看开头的游戏，每一行都能讲清楚了：

```
1 import random # 借用标准库来产生随机数
2
3 answer = random.randint(1, 100) # 1 到 100 之间随机取一个，两端都含
4 count = 0 # 记录猜了几次
5
6 while True: # 无限循环，靠 break 结束
7     guess = int(input("猜一个数: ")) # input 拿到字符串，int 转成整数
8     count = count + 1
9
10    if guess == answer: # 从上往下判断，猜中就结束
11        print(f"猜中了，你用了 {count} 次")
12        break
13    elif guess < answer:
14        print("小了")
15    else:
16        print("大了")
```

有两处细节值得留意。一是 `int(input(...))`，上一章说过 `input` 拿到的是字符串，不转成整数就没法和 `answer` 比大小。二是 `random.randint(1, 100)` 两端都包含，和 `range` 不含终点的规矩不一样，用的时候注意区分。

3.7 小结

这一章让程序会拐弯也会重复。分支用 `if elif else`，从上往下检查，谁先成立走谁，所以顺序有讲究。多条件用 `and or not` 连接。`for` 适合遍历一批数据或用 `range` 循环固定次数，`range` 从 0 开始不含终点。`while` 适合事先不知道多少次的情况，一定要保证条件能变假，否则死循环。`break` 跳出整个循环，`continue` 跳过本轮。列表推导式能把简单的加工循环压成一行。下一章把这个游戏拆成几个函数。

本章知识点

- **if elif else** —— 条件分支，从上往下检查，谁先成立执行谁，顺序有讲究
- **and or not** —— 连接多个条件，`and` 要求都成立，`or` 有一个成立即可
- **for** —— 遍历一批数据，每轮自动取出下一个元素
- **range** —— 产生一串数字用于循环，从 0 开始且不含终点
- **enumerate** —— 遍历时同时拿到序号和元素
- **while** —— 条件成立就一直循环，循环体里必须能让条件变假
- **死循环** —— 条件永远为真导致程序卡住，按 `Ctrl + C` 终止
- **break** —— 立刻跳出整个循环
- **continue** —— 跳过本轮剩下的部分，直接进入下一轮
- **列表推导式** —— 用一行从旧列表造出新列表，可以带筛选条件
- **import** —— 借用现成的库，比如 `random` 用来产生随机数